

LLM-based API Argument Completion with Knowledge-Augmented Prompts

Waseem Akram*
Beijing Institute of Technology
Beijing, China
waseemakramcui@gmail.com

Yanjie Jiang†
Tianjin University
Tianjin, China
yanjiejiang@pku.edu.cn

Haris Ali Khan
Beijing Institute of Technology
Beijing, China
hariskhatter@gmail.com

Furqan Jalil
Beijing Institute of Technology
Beijing, China
furqanjalil90@bit.edu.cn

Hui Liu
Beijing Institute of Technology
Beijing, China
liuhui08@bit.edu.cn

Abstract

Application Programming Interfaces (APIs) are fundamental to modern software development, yet accurately completing API method arguments remains a challenging task. Existing methods, such as information retrieval and hybrid program analysis with language models, often struggle with static code patterns and limited domain-specific knowledge. To this end, in this paper, we introduce APICopilot, a novel approach that enhances LLM-based API argument completion using dynamically generated, context-rich prompts. First, it creates an input knowledge graph (KG) from the preceding code for the to-be-complete API invocation, and a targeted knowledge graph from a collection of similar code files, both representing the anticipated API call. Second, it quantifies structural and semantic alignment between the two KGs and retrieves relevant subgraphs from the targeted graph. These subgraphs provide crucial structural context for in-context learning, enabling significantly more accurate and context-specific argument completions. Finally, we integrate the subgraphs into prompts and request LLMs to recommend API arguments. Our evaluation results on 54,000 Java files suggest that the proposed approach outperforms the state-of-the-art approach, improving precision, recall, and mean reciprocal rank by 38%, 36% and 23%, respectively.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments.**

Keywords

Code Completion, LLM, Knowledge Graph, API

ACM Reference Format:

Waseem Akram, Yanjie Jiang, Haris Ali Khan, Furqan Jalil, and Hui Liu. 2026. LLM-based API Argument Completion with Knowledge-Augmented

*Waseem Akram is also with Department of Computer Science, COMSATS University Islamabad, Vehari Campus, Pakistan

†Corresponding Author: Yanjie Jiang



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773158>

Prompts. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773158>

1 Introduction

Application Programming Interfaces (APIs) are fundamental to modern software development, enabling reuse of predefined components. However, effectively utilizing APIs remains challenging due to their complexity and evolving nature. Existing techniques, while assisting with method recommendation [15, 42], code synthesis [28, 29], and parameter analysis [9, 58], often neglect the crucial task of completing API arguments (actual parameters). Empirical studies show that over half of method calls require non-trivial arguments [59], and improper argument usage leads to bugs, security flaws, and inefficiencies [36].

To alleviate manual editing of API arguments, a few approaches have been proposed to automatically suggest arguments [9][41]. Asaduzzaman et al. [9] utilized information retrieval (IR) to map contextual features to historical usage, relying on a database of argument-context pairs and static features like method names and preceding code tokens. However, this method is limited by its inability to generate novel arguments, as it only recommends previously observed instances, failing to capture evolving API usage and nuanced contextual variations. Nguyen et al. [41] advanced this field by integrating program analysis with language models for improved syntactic and contextual understanding. Nevertheless, their focus on in-code patterns overlooks valuable information from external documentation, such as API specifications and usage examples, potentially limiting recommendation accuracy and context relevance.

Despite advancements in API argument completion, a significant challenge remains in accurately handling runtime-dependent arguments, whose precise values are determined only during program execution, making them unknowable through static analysis alone [53, 55]. These can include anything from user input and data fetched from external sources like databases or APIs, to system-generated values or results from complex runtime computations. For many code assistance tools, this dynamic nature creates a hurdle: without a statically known value, it becomes difficult to identify the correct or most relevant argument, especially when a variable's name or static type offers insufficient clues, or when arguments undergo intricate data flow transformations. This critical limitation underscores the necessity for a more dynamic, knowledge-rich,

and adaptable approach to API argument completion. Furthermore, while general-purpose Large Language Models (LLMs) excel at generating broad code snippets [48], they often struggle with the precise, context-dependent details essential for accurate API argument completion, frequently hallucinating incorrect arguments, misinterpreting semantic requirements, or suggesting syntactically valid but functionally erroneous values [27, 37].

To address these challenges, we introduce APICopilot, a novel approach to LLM-based API argument completion. APICopilot leverages KG-based RAG (Retrieval-Augmented Generation) to generate context-rich prompts. Unlike approaches relying on static KGs and retrieving noisy or irrelevant information [11][33], APICopilot employs a two-stage process: Dynamic KG generation and graph matching. Dynamic KG generation creates an input KG from the preceding code of the to-be-complete API invocation and a targeted KG from contextually similar code examples. Graph matching employs graph isomorphism and node-edge resonance proximity to retrieve highly relevant subgraphs from the targeted KG. By incorporating these pertinent subgraphs, which represent relevant data flow of API usage, into the LLM's prompt, APICopilot enables the LLM to generate context-specific argument completions. The APICopilot addresses the complexities of runtime-dependent arguments. This is achieved by understanding variables and their origins within the current code, and then leveraging similar patterns from a knowledge graph to construct an informative prompt for an LLM. This process enables the LLM to accurately infer and recommend the correct variable values. Illustrative examples demonstrating this capability are available in our online repository [4]. Our evaluation results on 54,000 Java files suggest that the proposed approach outperforms the state-of-the-art approach, improving precision, recall, and mean reciprocal rank by 38%, 36% and 23%, respectively. This paper makes the following contributions:

- A novel approach to API argument recommendation. To the best of our knowledge, we are the first to leverage knowledge graph-based RAG for API argument recommendation.
- Implementation and evaluation of the proposed approach. The replication package is publicly available [6].

2 Related Work

This approach builds on insights from several influential approaches, including ARist [41], PARC [9], and Precise [58]. ARist employs program analysis (PA) and fine-tuned language models (LMs) to generate syntax-valid API arguments, achieving notable improvements over earlier methods. However, its reliance on in-code patterns often overlooks external documentation or security policies, and its LM-based ranking inherits biases from training data, limiting robustness for runtime-dependent arguments like environment variables. PARC and Precise, conversely, adopt information retrieval (IR) techniques to identify similar arguments from a corpus. PARC employs SimHash for static features, while Precise uses k-nearest neighbors. IR-based methods recommend arguments by matching predefined static context features against a fixed, pre-indexed corpus. This reliance on previously observed arguments inherently limits their ability to suggest novel arguments and leads to shortcomings in ‘dynamic contexts’ (e.g., evolving API usage,

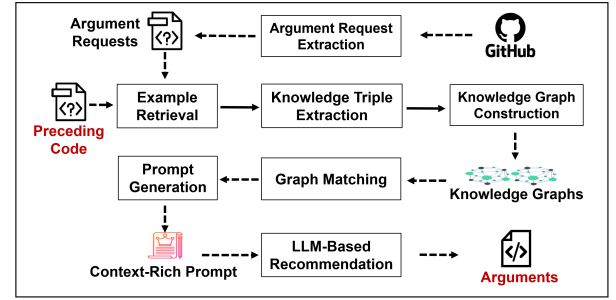


Figure 1: Overview of APICopilot

new code patterns) or ‘domain-specific contexts’ (e.g., specialized libraries). APICopilot overcomes these challenges by integrating semantic retrieval with dynamic, context-sensitive knowledge graph construction and LLM-driven generation.

Modern IDEs like Eclipse [24] and IntelliJ IDEA [30] provide basic API suggestions using program analysis (e.g., type compatibility). However, merely listing valid candidates alphabetically often fails to capture developer intent. To address this, researchers have prioritized ranking and filtering APIs using historical usage patterns. Recently, neural approaches like DeepAPI [25] and Codex [15] leverage sequence modeling for method recommendation but lack explicit argument dependency modeling. Huang et al. [29] highlight a critical gap: task-API knowledge mismatches. Yet, none of these methods systematically address argument completion. Recent advancements in LLMs highlight their growing proficiency in code completion, spanning synthesis, repair, and explanation [39][17][60]. However, direct application for specific tasks, e.g. API argument completion, faces challenges: fine-tuning is resource-intensive [52], and reliance on internal knowledge risks hallucinations. RAG reduces hallucinations by grounding LLMs in external data [11], but traditional chunk-based retrieval (vector embedding + similarity search) lacks structural information [23]. Knowledge graph-based RAG (GraphRAG) mitigates this issue through structural information retrieval [14, 23]. KGs encode structured relationships [31], enabling the retrieval of domain-specific constraints critical for accurate argument inference.

Different from KG-GPT [33], which leverages LLMs to filter relations within large KGs, our approach exclusively uses LLMs to generate a concise pattern graph. Furthermore, our method fundamentally differs from KAPING [11] by explicitly constraining subgraphs to be both structurally and semantically aligned with the pattern. This capability allows the proposed approach to support more complex and precise pattern graph structures.

3 Approach

3.1 Overview

Figure 1 presents an overview of our approach. Given the preceding code with missing arguments, it retrieves similar code examples and extracts knowledge triples from them. These triples are used to construct KGs, and the approach leverages graph matching to identify syntactically and semantically similar subgraphs. These subgraphs and retrieved examples are incorporated into a prompt for LLM that generates the suggested arguments.

```

1 public class ImageEditor {
2     public BufferedImage crop(BufferedImage image, int x,
3         int y, int width, int height) { //definition... }
4     public static void main(String args) throws
5         IOException {
6         BufferedImage specialImage = ImageIO.read(new
7             File("path/to/special.gif"));
8         ImageEditor editor = new ImageEditor();
9         BufferedImage croppedImage = editor.crop(
10             specialImage, 100, 100, 200, 200);
11         //Subsequent code...

```

Listing 1: Example of Argument Request

```

1 Image originalImage = new Image("path/to/image.jpg");
2 ImageTransformer transformer = new ImageTransformer();
3 Image resizedImage = transformer.resize(originalImage, /*
4     Missing Arguments

```

Listing 2: Input Preceding Code

3.2 Argument Request Extraction

The Argument Request Extraction (ARE) performs a preliminary code analysis to initiate the API argument completion process. This step identifies and extracts the relevant code context needed to complete API argument specifications accurately. By focusing on individual argument requests within a code file, ARE prepares the essential code elements crucial for data flow API usage for subsequent processing by APICopilot’s core components. In the context of API argument completion, “*data flow API usage*” refers to the comprehensive tracking of how data (or values) originates, propagates through different variables and expressions, and undergoes transformations within the code before finally being passed as an argument to an API call.

To illustrate the concept of AR, consider the code snippet in Listing 1. Given a method call `editor.crop()`, we extract the preceding code snippet P and the argument specifications $Args$ as follows:

$AR : \langle P: \text{preceding code snippet up to } editor.crop(),$
 $mcall: editor.crop(), Args: [(specialImage, 0),$
 $(100, 1), (100, 2), (200, 3), (200, 4)] \rangle$

Noted AR of complete code have fully specified $Args$, while AR of input preceding code have partially or fully missing $Args$. To make it clear, we define AR as follows:

Definition 3.1. An Argument Request (AR) is a tuple $AR = \langle P, mcall, Args \rangle$, Where:

- P The preceding code snippet serves as the method call’s contextual environment $mcall$.
- $mcall$ is the method call with one or more arguments to be completed.
- $Args$ is an ordered list of argument specifications: $Args = [(arg_1, pos_1), \dots, (arg_n, pos_n)]$, where: arg_i : Argument value or a placeholder (denoted as null if missing) and pos_i : The positional index of arg_i within $mcall$, starting from 0.

The ARE process systematically parses the code files, identifying $mcall$ with $Args$ within each .py or .java file. For each identified $mcall$, ARE employs scope analysis to extract the preceding code snippet P . This P provides the contextual information relevant to the $mcall$. The P and $mcall$, along with its $Args$, form an AR tuple.

```

1 // Example 1
2 Image anotherImage = new Image("path/to/another.png");
3 ImageTransformer anotherTransformer = new
4     ImageTransformer();
5 Image resizedAnotherImage = anotherTransformer.resize(
6     anotherImage, 200, 150);
7 // Example 2
8 Image myImage = new Image("path/to/my.jpeg");
9 ImageTransformer myTransformer = new ImageTransformer();
10 // Note the reversed order of arguments
11 myTransformer.resize(300, 200, myImage);

```

Listing 3: Retrieved Examples

ARE is implemented using Python’s `ast` [50] module for parsing .py files and a Java parser [1] for .java files. This implementation enables the systematic identification and extraction of AR triples from the code files of the training dataset.

3.3 Example Retrieval

Due to their fixed nature, static KGs often fail to capture the evolving landscape of API usage and diverse domain-specific knowledge. In contrast, dynamic and targeted KGs, constructed from similar code examples, provide run time and relevant information for accurate LLM-based argument completion. The Example Retrieval (ER) process necessitates retrieving a small set of highly relevant AR s and then their indexed knowledge triples to effectively build these targeted KGs [12]. To achieve this, we leverage the CodeLlama [44] code embedding model due to its strong performance i.e. 82.7% pass@1 accuracy on embedding benchmark MBPP [10].

Let $\mathcal{D}_{train}$ denote the complete set of argument requests in the training dataset. To identify relevant examples, ER calculates the Semantic Similarity Index (SSI) [18] between AR_{input} and each $AR_{training} \in \mathcal{D}_{train}$ as follows:

$$SSI(AR_{input}, AR_{training}) = \cos(\mathbf{z}_{input}, \mathbf{z}_{training}) \quad (1)$$

where $\mathbf{z}_{input} = \text{embed}(AR_{input})$ and $\mathbf{z}_{training} = \text{embed}(AR_{training})$ are code embeddings generated using the embedding model. The cosine similarity is computed as:

$$\cos(\mathbf{z}_{input}, \mathbf{z}_{training}) = \frac{\mathbf{z}_{input} \cdot \mathbf{z}_{training}}{\|\mathbf{z}_{input}\| \|\mathbf{z}_{training}\|} \quad (2)$$

After computing the SSI for AR_{input} against all $AR_{training}$ instances in $\mathcal{D}_{train}$, we obtain a ranked list of pairs $(AR_{training}, SSI)$. We then select the Top-K $AR_{training}$ instances with the highest SSI scores to form the set of example argument requests, $AR_{example}$. This selection process can be formally expressed as:

$$AR_{example} = \text{Top-K}_{SSI}(\{AR_{training} \mid AR_{training} \in \mathcal{D}_{train}\}) \quad (3)$$

For a given AR_{input} (as shown in Listing 2), the ER process retrieves similar $AR_{example}$ (as presented in Listing 3). We empirically evaluated the impact of the size (Top-K) of the set of best examples on the overall performance of APICopilot in Sect. 4.6.1. These AR s serve as the foundation for subsequent steps.

3.4 Knowledge Triple Extraction

The Knowledge Triple Extraction (KTE) transforms the raw code of AR s into knowledge triples. Inspired by Cai et al.’s demonstration of LLMs’ effectiveness in aligning queries to graph patterns [13], the process gives the following prompt to the ChatGPT-4o to analyze

```
(originalImage, typeOf, Image)
(originalImage, hasValue, "path/to/image.jpg")
(transformer, typeOf, ImageTransformer)
(resizedImage, assignedFrom, transformer.resize)
(transformer.resize, takesArgument, originalImage)
(transformer.resize, takesArgument, null) % Missing Arguments
(transformer.resize, takesArgument, null) % Missing Arguments
```

Listing 4: Knowledge Triples of Input Argument Request

```
// Triples of Example 1
(anotherImage, typeOf, Image)
(anotherImage, hasValue, "path/to/another.png")
(anotherTransformer, typeOf, ImageTransformer)
(resizedAnotherImage, assignedFrom, anotherTransformer.resize)
(anotherTransformer.resize, takesArgument, anotherImage)
(anotherTransformer.resize, takesArgument, 200)
(anotherTransformer.resize, takesArgument, 150)
// Triples of Example 2
(myImage, typeOf, Image)
(myImage, hasValue, "path/to/my.jpeg")
(myTransformer, typeOf, ImageTransformer)
(myTransformer.resize, takesArgument, 300)
(myTransformer.resize, takesArgument, 200)
(myTransformer.resize, takesArgument, myImage)
```

Listing 5: Knowledge Triples of Retrieved Examples

AR and identifies key entities (e.g., methods, variables, types) and their relationships (e.g., ‘calls’, ‘typeOf’, ‘takesArgument’, etc.):

“As an expert in code understanding, analyze the following code snippet: argument request. Extract knowledge triples (subject, predicate, object) that describe the API call relationships within this code. Focus on capturing API calls, arguments, argument types, and relationships crucial for understanding API usage. Output each extracted triple on a new line, formatted as: (Subject, Predicate, Object).”

These are then represented as triples, (h, r, t) , where h is the head entity, r is the relationship, and t is the tail entity. All such triples extracted from an AR constitutes a knowledge triple set : $(h_1, r_1, t_1), \dots, (h_n, r_n, t_n)$. This stage culminates in the generation of structured knowledge triple sets representing the essential entities and relationships within each AR. These sets are building blocks of the targeted KG, and are passed to the proceeding phase. The extracted knowledge triples of AR_{input} and $AR_{example}$ are presented in Listings 4 and 5.

3.5 Knowledge-Graph Construction

This phase constructs KGs from the knowledge triple sets generated in the KTE phase. It takes the knowledge triple sets of best example Argument Requests ($AR_{example}$) to construct a KG, $KG_{examples}$. Each triple (h, r, t) within these knowledge triple sets directly contributes to the graph structure. The construction of $KG_{examples}$ is defined as the union of all triples from the example ARs:

$$KG_{examples} = \bigcup_{i=1}^n (h_i, r_i, t_i)_{examples} \quad (4)$$

Here, i represents the index of triples ranging from 1 to n , and the subscript ‘examples’ denotes that these are relationships extracted from the best example ARs.

Similarly, a KG, G_{input} , is constructed from the knowledge triple set derived from the input Argument Request (AR_{input}):

$$G_{input} = \bigcup_{i=1}^n (h_i, r_i, t_i)_{input} \quad (5)$$

In both constructions, for each triple (h, r, t) , we create a directed edge in the KG originating from a node representing the head entity h , directed towards a node representing the tail entity t , and label this edge with the relation r . This process is repeated for all triples in the respective triple sets to fully materialize $KG_{examples}$ and G_{input} . These constructed KGs, $KG_{examples}$ and G_{input} , are essential inputs for the subsequent Graph Matching process. The Graph Matching phase leverages these graph structures to precisely identify relevant subgraphs within $KG_{examples}$ that exhibit structural similarity to G_{input} , as will be detailed in the following section.

3.6 Graph Matching

The Graph Matching (GM) identifies semantically similar subgraphs of $KG_{example}$ that match the G_{input} . This retrieval operates through two sequential phases: structural verification via graph isomorphism, and semantic evaluation using NERP.

3.6.1 Graph Isomorphism Filtering. The first phase of Graph Matching focuses on structural verification, aiming to identify subgraphs within $KG_{example}$ that are structurally equivalent to the input graph G_{input} . To establish structural equivalence, we employ the principle of graph isomorphism [21]. Two graphs are considered isomorphic if a bijective mapping exists between their vertex sets that preserve adjacency; that is, if two vertices are connected by an edge in one graph, their corresponding vertices are also connected in the other, and vice versa.

Algorithm 1 outlines our graph isomorphism filtering process. For the actual graph isomorphism verification within this process, we employ the efficient VF2 algorithm [19], which is grounded in backtracking principles. Algorithm 1 identifies structurally identical subgraphs from $KG_{examples}$ for a given input graph G_{input} . First, it extracts candidate subgraphs from the $KG_{examples}$ by iterating over its connected components (lines 2–5). For each component, it generates all possible subgraphs with node counts matching the G_{input} ’s size, ensuring candidates meet a necessary precondition for isomorphism. The algorithm then selects a root node r_{input} from the G_{input} (line 7) and iterates over all candidates (lines 8–13). For each candidate graph G_{cand} , it checks isomorphism by comparing every node rc as a potential root (lines 9–12) using a bijection-based verification (lines 14–18). If a match is found (line 10), the candidate is added to the result set $SubGraphs_{iso}$ (line 11), and the inner loop terminates early (line 12) to avoid redundant checks. This workflow ensures efficient pruning of non-isomorphic candidates through size-based filtering (line 15) and root-anchored structural validation (lines 17–18), returning all valid isomorphic subgraphs from the KG. This strict structural matching ensures $G_{candidate}$ share identical topology with G_{input} , forming the foundation for subsequent semantic analysis. In the graph isomorphism check (Algorithm 1), the subgraph of KG constituted by Triples Example 2 is omitted because, despite having the same number of nodes and edges as G_{input} , the order of arguments to `resize()` is different. This

Algorithm 1 Isomorphic Subgraph Filtering

Require: $G_{input} = (N_{input}, E_{input})$: Input graph
Require: $KG_{examples}$: Knowledge graph with example subgraphs
Ensure: $SubGraphs_{iso}$: List of isomorphic subgraphs

```

1: procedure GRAPHISOMORPHISM( $G_{input}, KG_{examples}$ )
2:    $candidates \leftarrow \emptyset$ 
3:   for each connected component  $C \in KG_{examples}$  do
4:     Generate all subgraphs  $S \subseteq C$  where  $|N_S| = |N_{input}|$ 
5:      $candidates \leftarrow candidates \cup \{S\}$ 
6:    $SubGraphs_{iso} \leftarrow \emptyset$ 
7:    $r_{input} \leftarrow \text{SELECTROOT}(G_{input})$ 
8:   for each  $G_{cand} = (N_c, E_c) \in candidates$  do
9:     for each node  $r_c \in N_c$  do
10:      if ISOMORPHIC( $G_{input}, G_{cand}, r_{input}, r_c$ ) then
11:         $SubGraphs_{iso} \leftarrow SubGraphs_{iso} \cup \{G_{cand}\}$ 
12:      break
13:   return  $SubGraphs_{iso}$ 

14: function ISOMORPHIC( $G_1 = (N_1, E_1), G_2 = (N_2, E_2), r_1, r_2$ )
15:   if  $|N_1| \neq |N_2| \vee |E_1| \neq |E_2|$  then
16:     return false ▷ Quick structural check
17:   Find bijection  $f: N_1 \rightarrow N_2$  where:
18:    $\forall u, v \in N_1 : (u, v) \in E_1 \iff (f(u), f(v)) \in E_2$ 
19:   return true if bijection exists, false otherwise

```

prevents a valid bijective mapping, as the relationships between elements are not preserved.

3.6.2 Node-Edge Resonance Proximity. NERP quantifies the relational similarity between graphs based on structural and contextual alignment, evaluating isomorphic graphs using Equation 6 (Definition 3.2). This metric (idea derived from SSI [18]) combines node-level and edge-level similarities through vector space embeddings, derived using CodeLlama-7B [44]. For each relation in the graphs, embeddings are calculated by comparing the head node from the input graph with the head node from the example graph and the tail node from the input graph with the tail node from the example graph. For edges, embeddings are produced by a relation-aware encoder that concatenates the head node, edge type, and tail node before feeding the sequence into CodeLlama. These embeddings explicitly model the syntactic and semantic relationships in code, as required by NERP's node-edge resonance framework. NERP then calculates semantic relevance by summing the cosine similarities of corresponding node and edge pairs across isomorphic subgraphs.

Definition 3.2. The Node-Edge-Resonance-Proximity (NERP) is a metric which measures the semantic relevance between isomorphic graphs G_a and G_b using the embeddings of their nodes and edges:

$$\text{NERP} = \underbrace{\sum_{v \in V_a} \cos(z_v, z_{f(v)})}_{\text{Node Proximity}} + \underbrace{\sum_{e \in E_a} \cos(z_e, z_{f(e)})}_{\text{Edge Proximity}} \quad (6)$$

where: z_v and $z_{f(v)}$ present the node embeddings in G_a and G_b . z_e and $z_{f(e)}$ are edge embeddings and $\cos()$ calculates the similarity.

```

<anotherImage,originalImage,0.95>
<anotherImagetypeOfImage,originalImagetypeOfImage,0.83>
<Image,Image,0.99>
...
// up to missing argument
<anotherTransformer.resize,transformer.resize,0.89>
<anotherTransformer.resizetakesArgument150,transformer.
  resizetakesArgumentnull,0.82>
<150, null,0.75>

```

Listing 6: NERP Calculation Between Graphs

Knowledge Triples of Input Graph:

```

(originalImage, typeOf, Image)
(originalImage, hasValue, "path/to/image.jpg")
(transformer, typeOf, ImageTransformer)
(resizedImage, assignedFrom, transformer.resize)
(transformer.resize, takesArgument, originalImage)
(transformer.resize, takesArgument, null)
(transformer.resize, takesArgument, null)

```

Knowledge Triples of Subgraph 1:

```

(anotherImage, typeOf, Image)
(anotherImage, hasValue, "path/to/another.png")
(anotherTransformer, typeOf, ImageTransformer)
(anotherTransformer.resize, takesArgument, anotherImage)
(anotherTransformer.resize, takesArgument, 200)
(anotherTransformer.resize, takesArgument, 150)

```

Knowledge Triples of Other Subgraphs ...

Best Examples...

Query: You are a code completion assistant. Analyze the knowledge triples of structurally and semantically similar graphs to the graph of the following input's preceding code to complete the missing arguments:

```

Image originalImage = new Image("path/to/image.jpg");
ImageTransformer transformer = new ImageTransformer();
Image resizedImage = transformer.resize(originalImage,

```

Figure 2: Example of Generated Prompt

Listing 6 illustrates the NERP calculation, showing the relationship between the knowledge triple (originalImage, typeOf, Image) from G_{input} and (anotherImage, typeOf, Image) from $KG_{example}$. Each line represents a comparison between corresponding nodes or edges in the two graphs, presented as a tuple: (node or edge of $KG_{example}$, node or edge of G_{input} , Score). The best subgraphs are selected through top-k NERP maximization:

$$TopGraphs = \arg \max_{KG_{subgraph} \subseteq KG_{examples}} \text{Top-K (NERP)} \quad (7)$$

We empirically evaluated the impact set size (Top-K) of subgraphs on overall APICopilot performance in Sect. 4.6.3. The final phase transforms $TopGraphs$ into machine-readable triples for downstream analysis. These triples enable explicit relationship modeling between code components, providing structured input for API parameter prediction and program analysis. The triple format facilitates pattern matching against knowledge bases and supports reasoning about entity interactions.

3.7 Prompt Generation

This phase constructs an LLM-readable prompt incorporating contextual information from previous stages. The prompt includes: (1) knowledge triples of G_{input} and $TopGraphs$, (2) best examples (ARs), (3) the query, and (4) the input preceding code. These are structured into a prompt template with few-shot examples (Figure 2), guiding the LLM to predict missing arguments.

3.8 LLM-Based Argument Recommendation

The generated context-rich prompt (denoted as *Prompt*) is sent to an LLM. We employ *ChatGPT-4o* as our primary LLM due to its state-of-the-art code understanding capabilities. The LLM output contains recommended API arguments. Following the baseline [41], we perform a post-processing step to guarantee that the LLM's output aligns with the expected API argument types. This involves filtering out the LLM-generated arguments and removing any suggestions that are not type-compatible with the API method's parameter. For example, a method `updateSettings(String username, int maxConnections)` expects a string for the username and an integer for the maximum connections. If the LLM "high" for `maxConnections`, the post-processing step would remove this suggestion "high" is a string (not an integer), thus ensuring that the arguments match the expected types. While this type checking has limitations in scenarios with complex type hierarchies or dynamic typing, it provides a reasonable approximation of type safety. Note that an expected argument could be a literal or a lambda expression. AR tools might not be able to correctly recommend the whole expressions. Thus, if the approaches can correctly suggest the expression type, we consider it as a correct recommendation. Instead of suggesting incorrect ones, recommending default values (e.g., "" for string literals and 0 for number literals) or a template of the expected expression types would be more appropriate for these cases. Note that this post-processing procedure is consistently applied to recommendations of all applied approaches detailed in Section 4.2.

4 EVALUATION

4.1 Research Questions

This evaluation addresses the following research questions:

- **RQ1:** Does APICopilot outperform existing approaches in API argument recommendation?
- **RQ2:** What is the impact of individual component of APICopilot on overall performance?
- **RQ3:** How does the underlying LLM affect the performance of APICopilot?
- **RQ4:** How effective is APICopilot on new projects?

RQ1 concerns the performance of the approach, compared against the state of the art. RQ2 concerns the effect of the components within the proposed approach, whereas RQ3 concerns the effect of the underlying LLM. RQ4 concerns the approach's generalizability.

4.2 Baselines

For a comprehensive comparison, we take the following baselines:

- (1) **Task-Specific Approach:** To compare against the current state-of-the-art API argument completion, we include ARist [41], a

Table 1: Evaluation Dataset

	Java	Python
#Project	Eclipse & Netbeans	PY150
#Code Files	54,000	150,000
# Lines of Code	7.2M	15.0M
#AR Requests	700,696	861,856

model explicitly designed for this task. We execute the original implementation [43] of ARist for Java projects with its default configurations.

- (2) **Code Completion Models:** To assess the performance of general-purpose code completion models on the specific task of API argument completion, we include CodeT5+ [57] and UniX-coder [26], widely recognized and utilized models in code completion [2, 16, 38, 56]. We fine-tuned them on our training dataset for API argument completion.
- (3) **LLM-Based Approach:** To evaluate the ability of LLMs to perform argument completion with minimal task-specific training (few-shot learning), and acknowledging the dynamic LLM landscape, we selected ChatGPT-4o [45], Gemini 2.0 Flash [3], and Llama 3 70B [22] at the time of selection based on their established benchmarks, accessibility, and task-relevant strengths in speed, context, and coding. For these few-shot approaches, the LLMs were provided with retrieved examples as part of a Retrieval-Augmented Generation (RAG) strategy, drawing contextual patterns as described in Section 3.3. For a test item, both APICopilot and the few-shot approaches received the same set of best examples and query instruction (as shown in Figure 2). However, APICopilot uniquely augmented these prompts with knowledge triples from both G_{input} and $TopGraphs$, a critical enhancement not present in the few-shot baselines.

For the baseline implementations, we directly reused the original ARist [41] framework (open-source [8]), a method combining program analysis with language models. CodeT5+ and UniXcoder were fine-tuned for API argument completion, employing a sequence-to-sequence setup where the preceding code served as input and the arguments as the target output. For the generic LLMs (ChatGPT-4o, Gemini 2.0 Flash, and Llama 3 70B), we accessed them via their respective APIs and utilized a few-shot prompting strategy, embedding in-context learning examples from the training data directly into each test query. The prompt template is publicly available online at GitHub [5]. Detailed implementations and source code for all these baseline approaches are available online [6].

4.3 Dataset

We evaluate APICopilot on two datasets: the Eclipse and Netbeans projects (Java) [9] for direct comparison with baseline ARist and the PY150 dataset (Python) [51] for cross-language evaluation. Table 1 summarizes the key statistics of each dataset. To ensure a fair evaluation, we preprocessed all datasets by removing duplicate files, forks, and non-executable code (e.g., comments). We standardized code formatting using the Eclipse JDT (Java) and Python AST parsers. Following the established process of baseline [41], we extracted features such as method/function signatures, argument

metadata (types, positions), and API usage context to facilitate the evaluation. These features are used to represent the code and enable comparisons between different code snippets.

4.4 Evaluation Process and Matrices

For a fair comparison with prior work [41], we used a consistent 80% training, 10% testing, and 10% validation split for both our Java and Python datasets. Crucially, our dataset splitting methodology ensures that the test set does not overlap with the training or validation sets, preventing data leakage and ensuring an unbiased evaluation of generalization performance. The test sets for Java and Python comprised 70,070 and 86,186 ARs, respectively, covering 7,500 distinct API calls in Java and 9,200 in Python. On average, each argument request in these test sets required the completion of 2.5 arguments for Java and 2.3 for Python. Baseline models (ARist, CodeT5+, UniXcoder) were trained/fine-tuned on the training data and evaluated on the testing data. For the few-shot approach and APICopilot, in-context learning examples (few-shot) are extracted from the training dataset and plugin prompt for each test item of the testing dataset. Noting that LLMs inherently possess randomness [34, 54], we trial each prompt five times for both the few-shot setting and APICopilot (and present the average performance) to ensure a robust evaluation.

We evaluated APICopilot and our baselines using *precision*, *recall*, and *Mean Reciprocal Rank*. Precision and recall are frequently used in evaluation of code completion and recommendation [35] whereas Mean Reciprocal Rank is a standard metrics in information retrieval. Precision (P) quantifies the accuracy of recommendations:

$$P = \frac{\# \text{ correct recommendations}}{\# \text{ generated recommendations}} \quad (8)$$

where a recommendation is *correct* if and only if the recommended argument is identical to the ground truth. Recall (R) measures the completeness or coverage of recommendations:

$$R = \frac{\# \text{ correct recommendations}}{\# \text{ arguments that should to be completed}} \quad (9)$$

Mean Reciprocal Rank (MRR) assesses ranking quality, crucial for quickly finding correct answers. For a set of queries Q , MRR is the average of reciprocal ranks for the first correct answer:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (10)$$

where rank_i is the position of the first correct recommendation for the i -th request. If no correct recommendation is made, its reciprocal rank is 0. A higher MRR directly improves developer efficiency and user experience by placing correct suggestions higher in the list.

4.5 RQ1: Improving the State of the Art

To address RQ1, we rigorously evaluated APICopilot against existing state-of-the-art approaches for API argument completion tasks. The evaluation results are presented in Table 2. The first column categorizes the datasets, specifically "Java" and "Python." Subsequent columns detail the performance of various approaches: APICopilot, ARist, CodeT5+, UniXcoder, ChatGPT, Gemini, and Llama—across the metrics of P , R and MRR . Table 2 demonstrates that APICopilot consistently outperforms ARist, a task-specific approach, across

```

1 public class APIClient {
2     public static void sendData(String url, String
3         authToken, String jsonPayload) {
4         HttpClient client = HttpClient.newHttpClient();
5         Map<String, String> headers = new HashMap<>();
6         headers.put("Authorization", "Bearer_" +
7             authToken);
8         headers.put("Content-Type", "application/json");
9         Duration timeout = Duration.ofSeconds(30);
10        HttpRequest request = HttpRequest.newBuilder()
11            .uri(URI.create(url))
12            .POST(/* Missing BodyPublisher */)
13            .timeout(/* Missing Timeout */)
14            .headers(/* Missing Headers Conversion */)
15            .build();

```

Listing 7: Example of Argument Request

all evaluation metrics for the Java dataset. For the Java dataset, APICopilot shows significant improvements over ARist, achieving a 37.71% $=((72.11\%-52.36\%)/52.36\%)$ relative improvement in precision and a 36.06% $=((69.46\%-51.05\%)/51.05\%)$ relative improvement in recall. Furthermore, in terms of ranking quality, APICopilot achieves a 23.44% $=((0.79-0.64)/0.64)$ relative improvement in MRR. This consistent advantage underscores APICopilot's superior effectiveness in accurately recommending API arguments compared to ARist for Java. ARist data for Python is not available in the table.

APICopilot substantially outperforms general code completion models (CodeT5+, UniXcoder) across all metrics (precision, recall, and MRR) in both Java and Python datasets, consistently achieving relative improvements exceeding 20% and often approaching or surpassing 30%. Specifically, for Java, it shows relative gains of 21.54% to 31.25% in precision, recall, and MRR over CodeT5+ and UniXcoder. For Python, these improvements are even stronger, ranging from 20.90% to 36.00%. Compared to LLM-based few-shot learning approaches (ChatGPT, Gemini, and Llama), APICopilot consistently demonstrates superior performance. For both Java and Python datasets, APICopilot achieves significant relative improvements in precision (18% to 25%), recall (18% to 25%), and MRR (9% to 15%) over these LLMs. These results collectively highlight APICopilot's sustained performance advantage across different model types and programming languages. We also evaluate all our approaches using *precision@K* and *recall@K* metrics defined by the authors of the baseline approaches, i.e., PARC [9] and ARist [41]. Concerning such metrics, APICopilot also substantially surpasses ARist by achieving a 30.31% $=((73.65\%-56.52\%)/56.52\%)$ relative improvement in precision@K and a 28.27% $=((70.91\%-55.28\%)/55.28\%)$ relative improvement in recall@K. For space limitations, details are presented online [7].

To reveal why the APICopilot can outperform the baselines, we manually analyzed some examples where it worked better than baselines. For input preceding code in Listing 7, all applied models complete the missing arguments shown in Listing 8. APICopilot's success stems from its retrieval of three critical subgraphs from the KG: (1) A *variable-to-API* subgraph linking `jsonPayload` $\rightarrow$ `BodyPublishers.ofString()` through the chain (`jsonPayload`, `usedIn`, `POST`), (`POST`, `requires`, `BodyPublisher`), (`BodyPublisher`, `instantiatedBy`, `ofString`), ensuring type-safe argument generation. (2) A *structural conversion* subgraph encoding `headers` $\rightarrow$ `entrySet().stream()` $\rightarrow$ `map(...)` $\rightarrow$

Table 2: Improving the State of the Art

Approaches	In Context Learning			Task-Specific Approach			Code Completion Models						LLM-Based (Few-Shot) Learning								
Datasets	APICopilot			ARist			CodeT5+			UniXcoder			ChatGPT			Gemini			Llama		
	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR
Java	72.11	69.46	0.79	52.36	51.05	0.64	54.94	52.99	0.65	56.85	54.74	0.67	60.38	58.26	0.72	57.53	55.48	0.69	59.57	57.36	0.71
Python	73.10	68.10	0.81	–	–	–	53.84	50.08	0.65	56.29	52.39	0.67	61.85	57.56	0.73	59.07	54.98	0.70	60.64	56.43	0.72

```

1 // Argument Completion By API Copilot
2 .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
3 .timeout(timeout) // Duration
4 .headers(headers.entrySet().stream()
5 .map(e -> new String[]{e.getKey(), e.getValue()}))
6 .flatMap(Arrays::stream)
7 .toArray(String[]::new)
8 // Argument Completion By Arist
9 .POST(HttpRequest.BodyPublishers.ofString(""))
10 .timeout(Duration.ofSeconds(10))
11 .header("Content-Type", "application/json")
12 //Argument Completion By Code T5+
13 .POST(HttpRequest.BodyPublishers.ofString("{}"))
14 .timeout(Duration.ofSeconds(10))
15 .headers("Content-Type", "application/json")
16 //Argument Completion By ChatGPT (few-shot)
17 .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
18 .timeout(timeout)
19 .headers(headers) // TYPE MISMATCH: Map vs. String[]
20 //Argument Completion By Gemini (few-shot)
21 .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
22 .timeout(Duration.ofSeconds(30))
23 .setHeader("Authorization", "Bearer_" + authToken)
24 //Argument Completion By Llama (few-shot)
25 .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
26 .timeout(timeout)
27 .headers("Authorization", "Bearer_" + authToken, "Content-Type", "application/json")

```

Listing 8: Arguments Completion by Models

toArray(String[]::new), which matches the isomorphic pattern (Map, convertedVia, stream), (stream, transformedBy, map), (map, flattenedBy, flatMap) observed in similar API usage examples. (3) A *temporal constraint* subgraph associating Duration timeout with .timeout() via the edge (timeout, typeOf, Duration), (timeout, passedTo, timeout). These subgraphs, ranked by Node-Edge Resonance Proximity (NERP), provide the LLM with syntactically valid and semantically grounded patterns for variable-aware completion.

ARist’s static patterns lack the Map→stream conversion subgraph, defaulting to hardcoded "" and header("Content-Type", ...). CodeT5+ prioritizes frequency-based literals ("") due to its training bias, missing the jsonPayload variable link. UniXcoder (not shown) fails similarly, suggesting invalid keySet() extraction. While ChatGPT infers variables correctly, its parametric knowledge lacks the stream-based subgraph, causing type-mismatched .headers(headers). Gemini retrieves deprecated .setHeader() from outdated API usage subgraphs in its training data. Llama replicates header literals verbatim, ignoring the structural Map→String[] subgraph. All baselines fail to retrieve and align the *exact conversion logic* (e.g., entrySet().stream()...)

Table 3: Impact of Size of Top-K Best Examples

Datasets	Metrics	K=30	K=50	K=100	K=150	K=200
Java	P(%)	68.37	71.17	72.11	72.13	72.15
	R(%)	65.46	68.46	69.46	69.48	69.50
	MRR	0.75	0.78	0.79	0.79	0.79
Python	P(%)	69.75	72.08	73.10	73.12	73.14
	R(%)	64.52	67.00	68.10	68.12	68.14
	MRR	0.77	0.80	0.81	0.81	0.81

that APICopilot’s KG grounding provides, as evidenced by their erroneous outputs in Listing 8.

4.6 RQ2: Ablation Study

To address RQ3, the ablation study investigates the impact of key components of the APICopilot on API parameter prediction.

4.6.1 Impact of Set Size of Best Examples. To assess the impact of the size of examples, we varied the number of top-K similar code examples retrieved during ER, adjusting the K value while keeping other system components at default settings.

The evaluation results are presented in Table 3. The first column categorizes the datasets ("Java" and "Python"), while the second lists evaluation metrics: Precision (P%), Recall (R%), and Mean Reciprocal Rank (MRR). Columns 3–7 show the performance of the model using varying sizes of the "Top-K" best examples (K=30, K=50, K=100, K=150, K=200) across both datasets. Table 3 shows that increasing the size of K from lower values to higher values generally leads to improvements in P, R, and MRR. For the Java dataset, moving from K=30 to K=100 results in a 5.47% relative improvement in P (from 68.37% to 72.11%), a 6.11% relative improvement in R (from 65.46% to 69.46%), and a 5.33% relative improvement in MRR (from 0.75 to 0.79). Similarly, in the Python dataset, increasing K from 30 to 100 yields a 4.80% relative improvement in P (from 69.75% to 73.10%), a 5.55% relative improvement in R (from 64.52% to 68.10%), and a 5.19% relative improvement in MRR (from 0.77 to 0.81). Notably, the improvements become marginal beyond K=100 for all metrics, indicating a point of diminishing returns. For instance, increasing K from 100 to 200 for Java yields only minor changes: P increases from 72.11% to 72.15% (0.06% relative improvement), R from 69.46% to 69.50% (0.06% relative improvement), and MRR remains at 0.79. This suggests that while a larger context window (higher K) initially provides more relevant information, beyond K=100, it yields only marginal improvements.

Table 4: Impact of Graph Matching

Configuration	Java			Python		
	P(%)	R(%)	MRR	P(%)	R(%)	MRR
GraphRAG (No GM)	65.37	62.25	0.71	66.33	59.77	0.72
GraphRAG + GM-Iso	69.60	66.03	0.76	70.59	64.92	0.78
GraphRAG + GM-Full	72.11	69.46	0.79	73.10	68.10	0.81
GM-Iso over GraphRAG	6.46	6.07	7.04	6.42	8.61	8.33
GM-Full over GraphRAG	10.31	11.58	11.27	10.21	13.94	12.50
GM-Full over GM-Iso	3.61	5.29	3.95	3.57	4.91	3.85

4.6.2 Impact of Graph Matching. To evaluate the impact of the GM module, we conducted experiments comparing performance across different GM configurations. We compared a baseline using "GraphRAG (No GM)" against variants: "GraphRAG + GM-Iso", which employs graph isomorphism filtering, and "GraphRAG + GM-Full", the default setting of APICopilot (which uses top-k syntactically and semantically similar subgraphs of $KG_{examples}$). This multifaceted approach isolates the contributions of GM's structural and semantic matching capabilities to overall system performance.

The evaluation results are presented in Table 4. The first column details the different configurations of the GraphRAG model, including "GraphRAG (No GM)", "GraphRAG + GM-Iso," and "GraphRAG + GM-Full," along with relative improvement metrics. Columns 2-7 show Precision (P%), Recall (R%), and Mean Reciprocal Rank (MRR) for both Java and Python datasets, demonstrating the impact of different graph matching techniques. Table 4 clearly demonstrates the positive impact of incorporating graph matching (GM) into the GraphRAG model. In both Java and Python datasets, the introduction of graph matching significantly enhances performance across all metrics. Specifically, "GraphRAG + GM-Iso," which utilizes isomorphism-based graph matching, shows notable improvements over the baseline "GraphRAG (No GM)." For Java, this configuration achieves a 6.46% relative improvement in P, a 6.07% relative improvement in R, and a 7.04% relative improvement in MRR. Similarly, for Python, the improvements are 6.42% in P, 8.61% in R, and 8.33% in MRR.

However, the most notable performance gains are observed with "GraphRAG + GM-Full," which employs a more comprehensive graph-matching strategy. This configuration outperforms both the baseline and "GraphRAG + GM-Iso." In the Java dataset, "GraphRAG + GM-Full" achieves a 10.31% relative improvement in P, an 11.58% relative improvement in R, and an 11.27% relative improvement in MRR compared to the "GraphRAG (No GM)" baseline. For Python, the improvements are 10.21% in P, 13.94% in R, and 12.50% in MRR. Furthermore, comparing "GraphRAG + GM-Full" to "GraphRAG + GM-Iso" reveals that the full graph matching approach provides additional gains. For Java, it shows improvements of 3.61% in P, 5.29% in R, and 3.95% in MRR. For Python, the additional gains are 3.57% in P, 4.91% in R, and 3.85% in MRR. These results underscore the importance of graph matching in enhancing the performance of GraphRAG. The full graph matching approach, in particular, demonstrates a substantial ability to improve Precision, Recall, and MRR, indicating that a more comprehensive comparison of graph

```

1 public class ListAdder {
2     public void addToList(List<String> myList, String
3         newItem) {
4         // Add newItem to myList
5         myList.???

```

Listing 9: Input Preceding Code

```

% Knowledge Triples of Input Preceding Code
(myList, isA, List)
(newItem, isA, String)
(myList, intendedAction, addElement)
(newItem, targetOf, addElement)
% Knowledge Triples of Retrieved Example
(names, isA, List)
("Alice", isA, String)
(names, addElement, "Alice") %<-- Direct match
(numbers, isA, List)
("One", isA, String)
(numbers, addElement, "One") %<-- Initial add
(num, isA, String) % (In loop context)
(num, usedIn, println)
(num, has, length)

```

Listing 10: Knowledge Triples

```

1 public static void main(String[] args) {
2     HikariConfig config = new HikariConfig();
3     config.setJdbcUrl("jdbc:mysql://localhost/db");
4     config.setUsername("user");
5     config.???;

```

Listing 11: Input Preceding Code

structures leads to more accurate, complete, and relevant recommendations. This highlights the effectiveness of leveraging graph-based representations for API argument completion tasks. To illustrate the importance of structural matching, we present an error analysis comparing GraphRAG (without GM) and GraphRAG + GM-Iso. Consider the task of predicting the correct method to add an element to a Java list, as shown in Listing 9. The goal is to predict the method call represented by 'myList.???'. We analyze the behavior of GraphRAG and GraphRAG + GM-Iso on constructed KGs presented in Listing 10. The GraphRAG considers all subgraphs, unable to distinguish the *context* of the addElement relationship – a direct addition in (names, addElement, "Alice") versus initialization (numbers, addElement, "One") before a loop. This can dilute confidence in the correct add() prediction or lead to an incorrect one. In contrast, GM-Iso omits the surrounding triples (representing a loop with println and length) of (numbers, addElement, "One") revealing a different structural context. Thus, GM-Iso correctly identifies (names, addElement, "Alice") as relevant, leading to a confident and accurate prediction of myList.add(newItem);. GraphRAG's error arises from considering matching triples in isolation, while GM-Iso's success is due to its ability to verify the *structural arrangement* of these triples and demonstrate the critical advantage of graph isomorphism. To illustrate the importance of full syntactical and semantically aliend GraphRAG, we presented an error analysis comparing GraphRAG (without GM) and GraphRAG + GM-Full. Listing 11, the goal is to predict the correct method to set the connection pool size for a

```
% Knowledge Triples of Input Preceding Code
(config, isA, HikariConfig)
(config, setJdbcUrl, "jdbc:mysql://localhost/db")
(config, setUsername, "user")
(config, intendedAction, Null)
(20, targetValue, setMaxPoolSize)
% Knowledge Triples of Retrieved Example
(prodConfig, isA, HikariConfig)
(prodConfig, setMaxPoolSize, 50)
(prodConfig, setJdbcUrl, "jdbc:mysql://production/db")
(prodConfig, setUsername, "prodUser")
(legacyConfig, isA, HikariConfig)
(legacyConfig, setMaximumConnectionPoolSize, 30)
(legacyConfig, setJdbcUrl, "jdbc:hsqldb://localhost/db")
(legacyConfig, setUsername, "sa")
```

Listing 12: Knowledge Triples

Table 5: Impact of Set Size of Top-K Subgraphs from GM

Datasets	Metrics	K=3	K=5	K=10	K=15	K=20
Java	P(%)	71.01	71.74	72.11	72.09	71.58
	R(%)	68.36	69.09	69.46	69.44	68.93
	MRR	0.78	0.79	0.79	0.79	0.78
Python	P(%)	72.00	72.73	73.10	73.08	72.57
	R(%)	67.00	67.73	68.10	68.08	67.57
	MRR	0.80	0.81	0.81	0.81	0.80

Table 6: Working with Various LLMs

LLM	Setting	Java			Python		
		P(%)	R(%)	MRR	P(%)	R(%)	MRR
ChatGPT	Few-Shot	60.38	58.26	0.72	61.85	57.56	0.73
	APICopilot	72.11	69.46	0.79	73.10	68.10	0.81
	Improvement	19.43	19.22	9.72	18.19	18.31	10.96
Gemini	Few-Shot	57.53	55.48	0.69	59.07	54.98	0.70
	APICopilot	71.10	68.45	0.77	72.30	67.30	0.79
	Improvement	23.59	23.38	11.59	22.39	22.41	12.86
Llama	Few-Shot	59.57	57.36	0.71	60.64	56.43	0.72
	APICopilot	69.86	69.86	0.78	71.93	71.93	0.79
	Improvement	17.27	21.79	9.86	18.62	27.47	9.72

HikariConfig object. The correct method is `setMaxPoolSize()`. From $KG_{example}$ presented in Listing 12, GraphRAG cannot distinguish between the correct `setMaxPoolSize` and the deprecated `setMaximumConnectionPoolSize`, potentially leading to the incorrect prediction: `config.setMaximumConnectionPoolSize(20);`. In contrast, GM-Full successfully identified that the surrounding triples of `(prodConfig, setMaxPoolSize, 50)` have a better NERF score than those of `(legacyConfig, setMaximumConnectionPoolSize, 30)`. This allowed GM-Full to correctly prioritize `(prodConfig, setMaxPoolSize, 50)`, leading to the accurate prediction of `config.setMaxPoolSize(20);`. The value 20 was intelligently inferred by the LLM as a more suitable default for the specific context, even though the retrieved example used 50.

4.6.3 Impact of Set Size of Top-k Subgraphs. To investigate the effect of varying the number of top-k subgraphs selected after GM-Full, we varied the number of K subgraphs while keeping other system components in default settings. The evaluation results are presented in Table 5, replicating the structural presentation of Table 3, while investigating the influence of varying subgraph set cardinalities ($K=3, 5, 10, 15, 20$) across Java and Python datasets. Table 5 shows that adjusting the size of K , representing the number of top- K subgraphs considered from graph matching, impacts performance. For the Java dataset, increasing K from 3 to 10 results in a 1.55% relative improvement in P (from 71.01% to 72.11%), a 1.61% relative improvement in R (from 68.36% to 69.46%), and a 1.28% relative improvement in MRR (from 0.78 to 0.79). Similarly, in the Python dataset, increasing K from 3 to 10 yields a 1.53% relative improvement in P (from 72.00% to 73.10%), a 1.64% relative improvement in R (from 67.00% to 68.10%), and a 1.25% relative improvement in MRR (from 0.80 to 0.81). However, the performance either stabilizes or declines slightly beyond $K=10$ for most metrics, indicating an optimal set size. For instance, for Java, P decreases from 72.11% at $K=10$ to 71.58% at $K=20$, and R decreases from 69.46% to 68.93%. This suggests that while a larger set of subgraphs initially provides more relevant information, an excessive number can introduce noise or redundancy, negatively affecting performance. The peak or stable performance at $K=10$ highlights the importance of selecting an appropriate set size for subgraph consideration in graph matching.

4.7 RQ3: Working with Various LLMs

To assess the impact of varying the underlying LLM, we replaced ChatGPT with mainstream LLMs Gemini-1.5 and Llama-3 and re-evaluated our approach. Table 6 presents the performance of various LLMs both in their native few-shot setting and when enhanced by APICopilot, highlighting the latter’s augmentation capabilities. Integrating APICopilot consistently and significantly improves the performance of all tested LLMs (ChatGPT-4o, Gemini-1.5, Llama-3) across both Java and Python datasets. ChatGPT shows P , R , and MRR gains of 19.43%, 19.22%, and 9.72% for Java, and 18.19%, 18.31%, and 10.96% for Python, respectively. Gemini achieves 23.59%, 23.38%, and 11.59% for Java, and 22.39%, 22.41%, and 12.86% for Python. Llama records 17.27%, 21.79%, and 9.86% for Java, and 18.62%, 27.47%, and 9.72% for Python. These substantial and consistent gains across diverse LLMs and datasets underscore APICopilot’s effectiveness and broad applicability in enhancing the quality of LLM-based argument recommendations for code completion.

4.8 RQ4: Performance on Unseen Project

To address potential data contamination, we constructed an unseen dataset by leveraging the knowledge cutoff date that ChatGPT-4o’s training data was limited to sources available before October, 2023 [46]. We compiled a small dataset of "Unseen" Java code from projects [6] and repositories published after this date, ensuring it was unseen by ChatGPT-4o [32]. This allowed for a more robust evaluation of generalization and mitigated the influence of potential overlap between our evaluation data and ChatGPT-4o’s training data [47][12][20]. For this evaluation, we used ChatGPT-4o as the underlying model for APICopilot and also evaluated ChatGPT

Table 7: Performance on Unseen Data

Approaches	In Context Learning			Task-Specific Approach			Code Completion Models						LLM-Based (Few-Shot) Learning								
	APICopilot			ARist			CodeT5+			UniXcoder			ChatGPT			Gemini			Llama		
Datasets	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR	P(%)	R(%)	MRR
Seen	72.11	69.46	0.79	52.36	51.05	0.64	54.94	52.99	0.65	56.85	54.74	0.67	60.38	58.26	0.72	57.53	55.48	0.69	59.57	57.36	0.71
Unseen	70.68	68.05	0.78	44.83	43.92	0.57	47.98	44.99	0.56	49.50	46.66	0.58	51.72	49.50	0.62	49.30	46.80	0.59	52.49	50.00	0.63
Drop (-%)	1.98	2.03	1.27	16.80	16.23	12.28	12.67	15.10	13.85	12.93	14.76	13.43	14.34	15.04	13.89	14.30	15.64	14.49	11.88	12.83	11.27

(few-shot) and fine-tuned CodeT5+ and UniXcoder on this Unseen data. We additionally include results for Gemini and Llama. The evaluation results are presented in Table 7. Table 7 shows that all models experience performance drops when evaluated on the 'Unseen' dataset compared to 'Seen' data. APICopilot demonstrates superior robustness to domain shifts with the smallest average decline: 1.98% in P, 2.03% in R, and 1.27% in MRR. This indicates strong generalization capabilities. In contrast, other approaches exhibit significantly larger performance drops. These results highlight APICopilot's ability to maintain high performance even when faced with data not present in its training or demonstration sets, showcasing its strong generalization and robustness compared to existing methods.

4.9 Threats to Validity

This research aimed to establish strong results regarding the performance of large language models in API argument inference. To address concerns about external validity, we evaluated various models, languages, and datasets, ensuring our findings were not limited to a specific context (in Section 4.5). Recognizing the potential for data contamination, a threat to internal validity, we incorporated an unseen dataset into our analysis to confirm the robustness of our results (in Section 4.8). Finally, we leveraged established API usage patterns from Netbeans, Eclipse and PY150 regarding construct validity (in Section 4.3). However, we acknowledge the inherent limitations in guaranteeing the absolute accuracy of the ground truth due to potential variations in optimal argument choices.

5 Discussion

Argument completion remains useful even in the era of LLMs. Even with the most advanced LLMs, only around 25% of Google's code and 30% of Microsoft's code are currently written by AI [40, 49]. This signifies that developers continue to write the majority of source code in a traditional manner, where robust API argument completion remains an essential aid. Another application scenario is to employ API argument completion tools to correct arguments generated by generic LLMs. As suggested by our evaluation, our approach is often more accurate than generic LLMs in suggesting arguments. Consequently, it is practical to replace arguments generated by LLMs with those suggested by our approach. Despite its effectiveness, APICopilot has certain trade-offs and limitations. Core components like argument request extraction, embedding of the training dataset for example retrieval, and the organization

of knowledge triples are pre-computed offline, reducing real-time load. However, online processes, namely dynamic knowledge graph construction for input, graph matching, and LLM inference, do introduce latency. As a result, the average response time per request is 2.83 seconds whereas the average response times for ARist and ChatGPT (few-shot) are 0.75 and 1.96 seconds, respectively. That is, APICopilot is comparable to baseline LLMs whereas ARist is substantially more efficient than LLM-based approaches. Furthermore, APICopilot's effectiveness is notably tied to the quality of its underlying knowledge graph. Its accuracy is inherently dependent on the relevance, diversity, and precision of the extracted knowledge triples from the code corpus, meaning suboptimal data can impact recommendations.

6 Conclusions

This paper introduced a novel approach that enhances LLM-based API argument completion by leveraging dynamically generated knowledge graphs and a graph-matching technique. To the best of our knowledge, this work represents the first successful application of knowledge graph-based RAG for API argument completion. The evaluations demonstrate that APICopilot significantly outperforms state-of-the-art approaches, including task-specific methods and general code completion models, achieving substantial improvements in precision and recall. APICopilot's effectiveness stems from its ability to provide LLMs with rich, context-specific information for in-context learning.

Acknowledgments

This work was partially supported by the National Natural Science Foundation of China (62232003 and 62172037).

References

- [1] [n. d.]. JavaParser. <https://javaparser.org/>. Accessed: 2025-02-28.
- [2] Waseem Akram, Yanjie Jiang, Yuxia Zhang, Haris Ali Khan, and Hui Liu. 2025. LLM-Based Method Name Suggestion with Automatically Generated Context-Rich Prompts. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE036 (June 2025), 22 pages. doi:10.1145/3715753
- [3] Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [4] APICopilot. 2025. ExplainatoryExample. [Online]. Available: https://github.com/copilotapiargument/APICopilotPaper/blob/main/Supplementary_Results/Answer%20of%20Q3.pdf. Accessed: March 14, 2025.
- [5] APICopilot. 2025. Generated Prompts by APICopilot. [Online]. Available: <https://github.com/copilotapiargument/APICopilotPaper/tree/main/GeneratedPrompts>. Accessed: March 14, 2025.

- [6] APICopilot. 2025. LLM-based API Argument Completion with Knowledge-Augmented Prompts. [Online]. Available: <https://github.com/copilotapiargument/APICopilotPaper>. Accessed: March 14, 2025.
- [7] APICopilot. 2025. SupplementaryResults. [Online]. Available: https://github.com/copilotapiargument/APICopilotPaper/blob/main/Supplementary_Results/Supplementary_Results.pdf. Accessed: March 14, 2025.
- [8] ARist. 2025. Source code of ARist. [Online]. Available: <https://github.com/ttrangnguyen/ARist>. Accessed: March 14, 2025.
- [9] Muhammad Asaduzzaman, Chanchal K Roy, and Kevin A Schneider. 2015. PARC: Recommending API methods parameters. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 330–332.
- [10] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [11] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. 2023. Knowledge-Augmented Language Model Prompting for Zero-Shot Knowledge Graph Question Answering. In *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE)*. Association for Computational Linguistics.
- [12] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [13] Yuzheng Cai, Zhenyue Guo, Yiwen Pei, Wanrui Bian, and Weiguo Zheng. 2024. SimGRAG: Leveraging Similar Subgraphs for Knowledge Graphs Driven Retrieval-Augmented Generation. *arXiv preprint arXiv:2412.15272* (2024).
- [14] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Wang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.
- [15] Chi Chen, Xin Peng, Zhenchang Xing, Jun Sun, Xin Wang, Yifan Zhao, and Wenyun Zhao. 2021. Holistic combination of structural and textual code information for context based API recommendation. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2987–3009.
- [16] Xiangpeng Chen, Xing Hu, Yuan Huang, He Jiang, Weixing Ji, Yanjie Jiang, Yanyan Jiang, Bo Liu, Hui Liu, Xiaochen Li, et al. 2025. Deep Learning-Based Software Engineering: Progress, Challenges, and Opportunities. *Science China Information Sciences* 68, 1 (2025), 1–88. doi:10.1007/s11432-023-4127-5
- [17] Yongchao Chen, Harsh Jhamtani, Srinagesh Sharma, Chuchu Fan, and Chi Wang. 2025. Steering Large Language Models between Code Execution and Textual Reasoning. In *The Thirteenth International Conference on Learning Representations*.
- [18] Moreno Colombo. 2024. *Semantic Similarity Measures*. Springer Nature Switzerland, Cham, 49–69. doi:10.1007/978-3-031-42819-7_4
- [19] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. VF2 algorithm revisited for subgraph isomorphism. *Int. J. Pattern Recognit. Artif. Intell.* 18, 08 (2004), 1373–1384. doi:10.1142/S021800140400377X
- [20] Common Crawl. 2024. *Open Repository of Web Crawl Data*. Retrieved 2024-07-27 from <https://commoncrawl.org/>
- [21] Reinhard Diestel. 2018. *Graph Theory* (5th electronic edition ed.). Graduate Texts in Mathematics, Vol. 173. Springer. <https://diestel-graph-theory.com/>
- [22] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [23] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 6491–6501.
- [24] Eclipse Foundation. 2018. Eclipse Code Recommenders. <http://www.eclipse.org/recommenders>. Online; Accessed 19 August 2021.
- [25] Xiaodong Gu, Hongyu Zhang, Dongmei Zhang, and Sunghun Kim. 2016. Deep API learning. In *Proceedings of the 2016 24th ACM SIGSOFT international symposium on foundations of software engineering*, 631–642.
- [26] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 7212–7225.
- [27] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.
- [28] Xincheng He, Lei Xu, Xiangyu Zhang, Rui Hao, Yang Feng, and Baowen Xu. 2021. Pyart: Python api recommendation in real-time. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1634–1645.
- [29] Qiao Huang, Xin Xia, Zhenchang Xing, David Lo, and Xinyu Wang. 2018. API method recommendation without worrying about the task-API knowledge gap. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 293–304.
- [30] JetBrains. 2021. IntelliJ IDEA. <https://www.jetbrains.com/help/idea/autocompleting-code.html>. Online; Accessed 21 August 2021.
- [31] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. 2021. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE transactions on neural networks and learning systems* 33, 2 (2021), 494–514.
- [32] Haris Ali Khan, Yanjie Jiang, Qasim Umer, Yuxia Zhang, Waseem Akram, and Hui Liu. 2025. Has My Code Been Stolen for Model Training? A Naturalness Based Approach to Code Contamination Detection. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE048 (June 2025), 22 pages. doi:10.1145/3715765
- [33] Jiho Kim, Yeonsu Kwon, Yohan Jo, and Edward Choi. 2023. KG-GPT: A General Framework for Reasoning on Knowledge Graphs Using Large Language Models. In *2023 Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics (ACL), 9410–9421.
- [34] Chris Lee et al. 2022. Temperature as a Control Mechanism for Diversity in Language Generation. *Transactions of the Association for Computational Linguistics* 10 (2022), 789–805.
- [35] Guangjie Li, Hui Liu, Ge Li, Sijie Shen, and Hanlin Tang. 2020. LSTM-based argument recommendation for non-API methods. *Science China Information Sciences* 63, 9 (2020), 190101. doi:10.1007/s11432-019-2830-8
- [36] Hui Liu, Qirong Liu, Cristian-Alexandru Staicu, Michael Pradel, and Yue Luo. 2016. Nomen est omen: Exploring and exploiting similarities between argument and parameter names. In *Proceedings of the 38th International Conference on Software Engineering*, 1063–1073.
- [37] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [38] Ehsan Mashhadi and Hadi Hemmati. 2021. Applying codebert for automated program repair of java simple bugs. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 505–509.
- [39] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro Von Werra, and Shayne Longpre. 2023. Octopack: Instruction tuning code large language models. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*.
- [40] Satya Nadella. 2025. *Satya Nadella says as much as 30% of Microsoft code is written by AI*. <https://www.cnbc.com/2025/04/29/satya-nadella-says-as-much-as-30percent-of-microsoft-code-is-written-by-ai.html>. Accessed: 2025-07-01.
- [41] Son Nguyen, Cuong Tran Manh, Kien T Tran, Tan M Nguyen, Thu-Trang Nguyen, Kien-Tuan Ngo, and Hieu Dinh Vo. 2023. Arist: An effective api argument recommendation approach. *Journal of Systems and Software* 204 (2023), 111786.
- [42] Son Nguyen, Hung Phan, Trinh Le, and Tien N Nguyen. 2020. Suggesting natural method names to check name consistencies. In *Proceedings of the acm/ieee 42nd international conference on software engineering*, 1372–1384.
- [43] Trang Nguyen. 2023. ARist: Source code and implementation. <https://github.com/ttrangnguyen/ARist>. Accessed: [Insert Date Here].
- [44] Ansong Ni, Pengcheng Yin, Yilun Zhao, Martin Riddell, Troy Feng, Rui Shen, Stephen Yin, Ye Liu, Semih Yavuz, Caiming Xiong, et al. 2024. L2ceval: Evaluating language-to-code generation capabilities of large language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 1311–1329.
- [45] OpenAI. 2023. GPT-4 Technical Report. <https://openai.com/research/gpt-4>.
- [46] OpenAI. 2024. *GPT-4o Turbo Updates*. Retrieved 2025-01-10 from <https://platform.openai.com/docs/models/gpt-4o>
- [47] Data Partnerships. 2023. *OpenAI Data Partnerships*. <https://openai.com/index/data-partnerships/>. Accessed: 2024-07-23.
- [48] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the keyboard? assessing the security of github copilot’s code contributions. *Commun. ACM* 68, 2 (2025), 96–105.
- [49] Sundar Pichai. 2024. *Google’s Code AI and the Future of Programming*. <https://fortune.com/2024/10/30/googles-code-ai-sundar-pichai/>. Accessed: 2025-07-01.
- [50] Python Software Foundation. 2023. *ast — Abstract Syntax Trees*. Python Software Foundation. <https://docs.python.org/3/library/ast.html>
- [51] Veselin Raychev, Pavol Bielek, and Martin Vechev. 2016. Probabilistic model for code with decision trees. *ACM SIGPLAN Notices* 51, 10 (2016), 731–747.
- [52] S. Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, J. Kepner, Devesh Tiwari, and V. Gadepally. 2023. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. *2023 IEEE High Performance Extreme Computing Conference (HPEC)* (2023), 1–9. doi:10.1109/HPEC58863.2023.10363447
- [53] Qi Shen, Shijun Wu, Yanzen Zou, and Bing Xie. 2021. Comprehensive integration of API usage patterns. In *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*. IEEE, 83–93.
- [54] John Smith and Alice Brown. 2023. Stochasticity in Neural Language Models: A Theoretical and Empirical Analysis. In *Advances in Neural Information Processing Systems*, Vol. 36. 12345–12360.
- [55] Ferdian Thung. 2016. API recommendation system for software development. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, 896–899.

- [56] Tim van Dam, Maliheh Izadi, and Arie van Deursen. 2023. Enriching source code with contextual data for code completion models: An empirical study. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 170–182.
- [57] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 1069–1088.
- [58] Cheng Zhang, Juyuan Yang, Yi Zhang, Jing Fan, Xin Zhang, Jianjun Zhao, and Peizhao Ou. 2012. Automatic parameter recommendation for practical API usage. In *2012 34th international conference on software engineering (ICSE)*. IEEE, 826–836.
- [59] Lei Zhang, Tong Wang, Shuang Wang, Tong Liu, Lin Tan, and Lan Bao. 2021. Automatic Detection and Correction of Inconsistent Method Names. *IEEE Transactions on Software Engineering* (2021). <https://doi.org/10.1109/TSE.2021.3050785>
- [60] Xin Zhou, Ting Zhang, and David Lo. 2024. Large Language Model for Vulnerability Detection: Emerging Results and Future Directions (*ICSE-NIER'24*). Association for Computing Machinery, New York, NY, USA, 47–51. doi:10.1145/3639476.3639762